

Inteligencia Artificial I

Laboratorio 2 · Búsqueda informada

La idea

Cada equipo recibe **una red de carreteras distinta**. El coste de cada tramo son **minutos**, no kilómetros. Y hay una ausencia deliberada: **no tenéis coordenadas, ni mapa a escala, ni distancias en línea recta**. Solo la tabla de conexiones.

Eso significa que la heurística que usa el libro —la distancia en línea recta hasta el destino— aquí no existe. **La tenéis que inventar vosotros**. Ese es el laboratorio.

Lo que se os entrega

- La hoja «**Red de carreteras · Equipo N**», con vuestras 28 ciudades, sus conexiones y la consulta que hay que resolver.
- El fichero **mapa_NN.csv** con los mismos datos, para el código.
- El fichero **verificar.py**, una herramienta de apoyo que comprueba vuestra heurística y mide su rendimiento. Leedlo antes de empezar: os ahorra la mitad del trabajo.

Las conexiones son de doble sentido. Cada una trae los kilómetros, el tipo de vía y los minutos que se tardan.

Tipo de vía	Velocidad
autovía	120 km/h
nacional	90 km/h
comarcal	60 km/h
tren lanzadera	160 km/h

Las reglas del juego son:

1. **$h(n)$ devuelve minutos**. Las mismas unidades que $g(n)$. Si mezcláis unidades, todo lo demás da igual.
2. **$h(\text{destino}) = 0$** .
3. **$h(n)$ no puede sobreestimar nunca**: $h(n) \leq h^*(n)$ para todas las ciudades del mapa.
4. **No vale usar Dijkstra desde el destino como heurística**. Eso es el oráculo: resuelve el problema que decís estar estimando. Sirve para comprobar, no para buscar.
5. **El precálculo sí está permitido**, pero hay que declararlo y contarlo como coste. Las heurísticas buenas casi siempre precalculan algo.

Fase 1 · La línea fundamental

Implementad A^* con una cola de prioridad (heapq) y conjunto de visitados. Ejecutadlo con **$h(n) = 0$** y anotad cuántos nodos expande.

Comprobaréis que expande las 28 ciudades. No es un fallo: con $h = 0$, **A*** es **exactamente el coste uniforme**. Ese número es vuestro punto de partida y el que tenéis que bajar.

Dos avisos que os van a ahorrar una hora de depuración:

- Al meter en el montículo tuplas (f , g , estado), si dos nodos empatan en f Python intentará comparar los estados. Meted un contador: (f , contador, estado).
- **A* no termina cuando genera el destino, sino cuando lo expande.** Si devolvéis el camino en cuanto veis el objetivo en la frontera, perdéis la optimalidad. Es el error número uno.

Fase 2 • Inventad vuestra heurística (lo fundamental)

Aquí está el trabajo de verdad. Pensadlo antes de programar: ¿qué sabéis sobre lo que falta para llegar, sin haber resuelto el problema?

Si os atascáis, pedidme las pistas. Hay tres, y van de menos a más.

Fase 3 • Medid

Rellenad esta tabla ejecutando las tres heurísticas sobre vuestra consulta. La tercera, h^* , la calcula `verificar.py`: es el límite que nadie puede batir.

	$h = 0$	Vuestra h	$h = h^*$
¿Admisible?			
¿Consistente?			
Nodos expandidos			
Nodos generados			
b^*			
Precisión media h/h^*			
Coste del camino (min)			

Las tres tienen que encontrar **el mismo coste**. Si la vuestra encuentra uno mayor, no es admisible: buscad la ciudad donde se pasa.

Fase 4 • Analizad

Contestad por escrito, con los números delante:

- **¿Vuestra heurística es consistente, además de admisible?** Si no lo es, ¿qué arista la rompe, y qué consecuencia tiene al usar lista de cerrados?
- **Dividid vuestra h entre dos.** ¿Sigue siendo admisible? ¿Expande más nodos o menos? ¿Cómo se llama eso que acabáis de comprobar?
- **¿Cuánto cuesta calcular vuestra h ?** Si precalculáis algo, ¿cuántas ejecuciones de Dijkstra son? ¿Compensa frente a los nodos que ahorra?

- **¿Dónde se queda corta?** Buscad la ciudad con peor precisión h/h^* y explicad por qué justo ahí.
- **Si vuestra heurística parte de los kilómetros**, mirad con atención la tabla de velocidades antes de dividir. ¿Por qué velocidad hay que dividir para no pasarse nunca?

El concurso (0.2 puntos) para los que quieran....no es obligatorio

Cada equipo ejecuta su heurística sobre su propio mapa y apunta dos números: **nodos expandidos** y **¿admisibile, sí o no?**

Gana quien menos nodos expanda. **Una heurística no admisibile queda descalificada**, por muy pocos nodos que expanda. La comprobación es automática y la hace `verificar.py` delante de todos.

Para que os hagáis una idea del listón: con $h = 0$ se expanden las 28 ciudades, y el oráculo —que es imbatible— se queda en torno a 10 u 11. Todo lo que hay entre medias es vuestro.

Entregables

1. Código Python con A* instrumentado y vuestra heurística.
2. La tabla de la Fase 3, rellena.
3. Las respuestas de la Fase 4 (máximo una página).
4. **Media página justificando por qué vuestra heurística nunca sobreestima.** No vale «lo hemos comprobado con el programa»: se pide el argumento.

Evaluación

Criterio	Peso
A* correcto, con visitados y montículo	25 %
Heurística admisibile y justificada con un argumento	30 %
Tabla de medidas completa y correcta	20 %
Análisis de la Fase 4	15 %
Calidad de la heurística (nodos expandidos)	10 %

Fijaos en el reparto: **justificar por qué es admisibile vale tres veces más que conseguir que expanda pocos nodos**. Una heurística mediocre bien argumentada saca mejor nota que una brillante que no sabéis explicar.

Anexo · Las tres pistas

Para la profesora: entregar de una en una, y solo a quien se atasque de verdad.

Pista 1 La más burda

Contad cuántos tramos faltan como mínimo hasta el destino (una BFS ignorando los costes) y multiplicadlo por el tramo más barato que exista en todo el mapa.

Es admisible, porque para recorrer k tramos hay que pagar al menos k veces el más barato de todos. Y apenas informa: comprobadlo y quedaos con el número, porque es la prueba de que «admisible» y «útil» no son lo mismo.

Pista 2 De kilómetros a minutos

Si conocéis los kilómetros mínimos que faltan, podéis convertirlos en minutos dividiendo por una velocidad. ¿Por cuál?

Tiene que ser una velocidad que **nadie pueda superar en ningún tramo del mapa**, o estaréis sobreestimando. Mirad la tabla de velocidades entera. Hay un medio de transporte más rápido que la autovía.

Pista 3 Puntos de referencia

Esta es la que usan los navegadores de verdad, y no necesita geometría ninguna.

Elegid unas pocas ciudades como **anclas** —tres bastan— y precalculad con Dijkstra la distancia real desde cada ancla a todas las demás ciudades. Entonces, para cualquier ciudad n :

$$h(n) = \text{máx sobre las anclas } L \text{ de } |d(L, n) - d(L, \text{destino})|$$

Es admisible por la desigualdad triangular: ir de n al destino no puede costar menos que la diferencia entre lo que cuesta llegar a cada uno desde una misma ancla. Escribid esa demostración, que es la mitad de la nota.

Y quedan dos preguntas abiertas que son las interesantes: **¿cuántas anclas?** y **¿dónde las ponéis?** Probad con anclas en el centro del mapa y con anclas en los extremos, y comparad. La diferencia es grande y no es evidente por qué.